

Instalar a CLI do NestJS globalmente (se já tiver, pode pular)

```
npm install -g @nestjs/cli
```

Criar um novo projeto NestJS

```
nest new nest-typeorm-sqlite --strict
```

```
cd nest-typeorm-sqlite
```

Instalar o TypeORM oficial do NestJS e o driver do SQLite

```
npm install @nestjs/typeorm typeorm sqlite3
```

1. Criando o Projeto NestJS e Instalando as Dependências

Primeiro, instale a CLI do NestJS (caso não tenha), gere um novo projeto e instale os pacotes necessários para o TypeORM rodar com o NestJS:

bash

Instalar a CLI do NestJS globalmente (se já tiver, pode pular)

```
npm install -g @nestjs/cli
```

Criar um novo projeto NestJS

```
nest new nest-typeorm-sqlite --strict
```

```
cd nest-typeorm-sqlite
```

Instalar o TypeORM oficial do NestJS e o driver do SQLite

```
npm install @nestjs/typeorm typeorm sqlite3
```

Use o código com cuidado.

2. Configurando o Banco no Módulo Principal ([src/app.module.ts](#))

No NestJS, usamos o `TypeOrmModule.forRoot()` no módulo raiz para estabelecer a conexão global com o banco de dados.

typescript

```
// src/app.module.ts
import { Module } from '@nestjs/common';
import { TypeOrmModule } from '@nestjs/typeorm';
import { User } from '../users/user.entity';
import { Photo } from '../photos/photo.entity';
import { UsersModule } from '../users/users.module';
import { PhotosModule } from '../photos/photos.module';
```

```
@Module({
  imports: [
    // Configuração global da conexão SQLite
    TypeOrmModule.forRoot({
      type: 'sqlite',
      database: 'database.sqlite',
      entities: [User, Photo], // Suas entidades globais
      synchronize: true,      // Apenas para ambiente de
desenvolvimento!
      logging: false,
    }),
    UsersModule,
    PhotosModule,
  ],
})
export class AppModule {}
```

Use o código com cuidado.

3. Criando o Módulo de Usuários (`Users`)

Podemos usar a CLI para gerar a estrutura de arquivos rapidamente:

bash

```
nest g module users
nest g service users
nest g controller users
```

Use o código com cuidado.

A. A Entidade (`src/users/user.entity.ts`)

A entidade permanece praticamente igual, mas geralmente a salvamos com a extensão `.entity.ts`:

typescript

```
// src/users/user.entity.ts
import { Entity, PrimaryGeneratedColumn, Column, OneToMany } from
'typeorm';
import { Photo } from '../photos/photo.entity';

@Entity()
export class User {
  @PrimaryGeneratedColumn()
  id: number;
```

```

@Column()
firstName: string;

@Column()
lastName: string;

@Column({ default: true })
isActive: boolean;

@OneToMany(() => Photo, (photo) => photo.user)
photos: Photo[];
}

```

Use o código com cuidado.

B. Registrando a Entidade no Módulo (`src/users/users.module.ts`)

Para que o NestJS saiba que este módulo usa a tabela `User`, precisamos importar `TypeOrmModule.forFeature([User])`. Isso torna o repositório disponível para injeção.

typescript

```

// src/users/users.module.ts
import { Module } from '@nestjs/common';
import { TypeOrmModule } from '@nestjs/typeorm';
import { UsersService } from './users.service';
import { UsersController } from './users.controller';
import { User } from './user.entity';

@Module({
  imports: [TypeOrmModule.forFeature([User])], // Define o repositório
  da entidade User
  providers: [UsersService],
  controllers: [UsersController],
  exports: [TypeOrmModule], // Exporta para que o PhotosModule possa
  associar fotos a usuários
})
export class UsersModule {}

```

Use o código com cuidado.

C. Injetando o Repositório no Serviço (`src/users/users.service.ts`)

Aqui está a grande mudança do NestJS: em vez de instanciar o `DataSource`, usamos o decorador `@InjectRepository` no construtor da classe. O NestJS cuida do resto.

typescript

```
// src/users/users.service.ts
import { Injectable } from '@nestjs/common';
import { InjectRepository } from '@nestjs/typeorm';
import { Repository, Like } from 'typeorm';
import { User } from '../user.entity';

@Injectable()
export class UsersService {
  constructor(
    @InjectRepository(User)
    private userRepository: Repository<User>, // Repositório injetado automaticamente
  ) {}

  // Método para criar um usuário de teste
  async createDemo() {
    const user = this.userRepository.create({
      firstName: 'Gabriel',
      lastName: 'Silva',
      isActive: true,
    });
    return this.userRepository.save(user);
  }

  // Busca Avançada A: Usuários ativos trazendo fotos
  async findActiveWithPhotos() {
    return this.userRepository.find({
      where: { isActive: true },
      relations: { photos: true },
    });
  }

  // Busca Avançada B: Filtro de texto parcial
  async findByLastName(partialName: string) {
    return this.userRepository.find({
      where: {
        lastName: Like(`%${partialName}%`),
      },
    });
  }
}
```

Use o código com cuidado.

D. Expondo as Rotas no Controller (`src/users/users.controller.ts`)

typescript

```
// src/users/users.controller.ts
import { Controller, Get, Post, Query } from '@nestjs/common';
import { UsersService } from '../users.service';

@Controller('users')
export class UsersController {
  constructor(private readonly usersService: UsersService) {}

  @Post('seed')
  async seed() {
    return this.usersService.createDemo();
  }

  @Get('active')
  async getActive() {
    return this.usersService.findActiveWithPhotos();
  }

  @Get('search')
  async search(@Query('lastName') lastName: string) {
    return this.usersService.findByLastName(lastName);
  }
}
```

Use o código com cuidado.

4. Criando o Módulo de Fotos (`Photos`)

Gere a estrutura da mesma forma:

bash

```
nest g module photos
nest g service photos
nest g controller photos
```

Use o código com cuidado.

A. A Entidade (`src/photos/photo.entity.ts`)

typescript

```
// src/photos/photo.entity.ts
```

```

import { Entity, PrimaryGeneratedColumn, Column, ManyToOne } from
'typeorm';
import { User } from '../users/user.entity';

@Entity()
export class Photo {
  @PrimaryGeneratedColumn()
  id: number;

  @Column()
  url: string;

  @Column()
  likes: number;

  @ManyToOne(() => User, (user) => user.photos, { onDelete: 'CASCADE'
})
  user: User;
}

```

Use o código com cuidado.

B. Configurando o Módulo de Fotos (`src/photos/photos.module.ts`)

Aqui nós importamos o repositório de `Photo` e também trazemos o `UsersModule` para que possamos buscar usuários se precisarmos vinculá-los às fotos.

typescript

```

// src/photos/photos.module.ts
import { Module } from '@nestjs/common';
import { TypeOrmModule } from '@nestjs/typeorm';
import { PhotosService } from './photos.service';
import { PhotosController } from './photos.controller';
import { Photo } from './photo.entity';
import { UsersModule } from '../users/users.module';

@Module({
  imports: [
    TypeOrmModule.forFeature([Photo]),
    UsersModule // Permite usar o repositório de User aqui dentro
  ],
  providers: [PhotosService],
  controllers: [PhotosController],
})
export class PhotosModule {}

```

Use o código com cuidado.

C. Injetando Ambos os Repositórios (`src/photos/photos.service.ts`)

Se você precisar criar uma foto vinculada a um usuário, pode injetar o `Repository<User>` e o `Repository<Photo>` no mesmo serviço:

typescript

```
// src/photos/photos.service.ts
import { Injectable, NotFoundException } from '@nestjs/common';
import { InjectRepository } from '@nestjs/typeorm';
import { Repository, MoreThan } from 'typeorm';
import { Photo } from '../photo.entity';
import { User } from '../users/user.entity';

@Injectable()
export class PhotosService {
  constructor(
    @InjectRepository(Photo)
    private photoRepository: Repository<Photo>,

    @InjectRepository(User)
    private userRepository: Repository<User>,
  ) {}

  // Criar foto vinculada a um usuário existente
  async addPhotoToUser(userId: number, url: string, likes: number) {
    const user = await this.userRepository.findOneBy({ id: userId });
    if (!user) throw new NotFoundException('Usuário não encontrado');

    const photo = this.photoRepository.create({ url, likes, user });
    return this.photoRepository.save(photo);
  }

  // Busca Avançada C: Filtro numérico com operadores (> X likes)
  async findPopularPhotos(minLikes: number) {
    return this.photoRepository.find({
      where: {
        likes: MoreThan(minLikes),
      },
      relations: { user: true },
    });
  }
}
```

Use o código com cuidado.

D. Criando o Controller de Fotos (`src/photos/photos.controller.ts`)

typescript

```
// src/photos/photos.controller.ts
import { Controller, Get, Post, Body, Query, ParseIntPipe } from
 '@nestjs/common';
import { PhotosService } from './photos.service';

@Controller(

  'photos')
export class PhotosController {
  constructor(private readonly photosService: PhotosService) {}

  @Post()
  async addPhoto(
    @Body('userId') userId: number,
    @Body('url') url: string,
    @Body('likes') likes: number,
  ) {
    return this.photosService.addPhotoToUser(userId, url, likes);
  }

  @Get('popular')
  async getPopular(@Query('minLikes', ParseIntPipe) minLikes: number) {
    return this.photosService.findPopularPhotos(minLikes);
  }
}
```

Use o código com cuidado.

5. Testando a API NestJS

Rode o projeto com o comando padrão do NestJS:

bash

```
npm run start:dev
```

Use o código com cuidado.

Agora você tem endpoints HTTP reais prontos para interagir com o TypeORM e SQLite através do seu client HTTP preferido (como Postman, Insomnia ou extension Rest Client):

1. **Criar um Usuário de Teste:** Envie um **POST** para <http://localhost:3000/users/seed>

Adicionar Foto para esse Usuário (ex: ID 1): Envie um `POST` para `http://localhost:3000/photos` com o JSON:

json

```
{ "userId": 1, "url": "viagem.jpg", "likes": 120 }
```

2. Use o código com cuidado.

3. **Buscar Usuários Ativos e Suas Fotos:** Envie um `GET` para `http://localhost:3000/users/active`

4. **Filtrar por Fotos Populares:** Envie um `GET` para `http://localhost:3000/photos/popular?minLikes=100`

●